

BASE HOME 项目素材库（论文写作的事实来源）

本文件是所有主题模板的底座。其余 01 ~ 09 号模板都从这里取素材。写作顺序建议：先读本文件 → 选定项目口径 → 再打开对应主题模板填充。

⚠ 先读这一节：素材可信度说明

论文最忌讳编造。本素材库严格区分两类内容：

标记	含义	处理方式
✔ 真实	来自项目源码/配置，可直接写进论文	放心使用
◆ 待你补充	项目里没有的数据（尤其是性能指标）	必须由你填入真实或合理数字，禁止直接照抄

重点提醒：项目源码中**不存在** QPS、响应时间、并发用户数、数据量等性能数据。论文要求「具体数字」，这部分**只能由你结合真实上线情况填写**。本文件在需要数字的地方一律标注 **◆**，不要编造后当成事实。

脱敏要求：论文中只能写「配置项名称 + 作用」，不得出现任何真实的 OAuth2 Client Secret、AWS Access Key、Cognito Issuer、数据库密码等值。

一、项目概况（论文第一段直接用）

项目名称：BASE HOME（ベースホーム）

所属企业：H.I.S.（エイチ・アイ・エス）——日本大型旅行社集团

业务定位：面向旅行者（旅行者）与营业本部（営業本部）的**旅游业务流程支撑平台**，覆盖预约管理（予約管理）、客户消息（顧客メッセージ）、通知发送（通知配信）、文件管理（ファイル管理）与主数据联动。

核心业务域（真实 Controller/Service 归类）

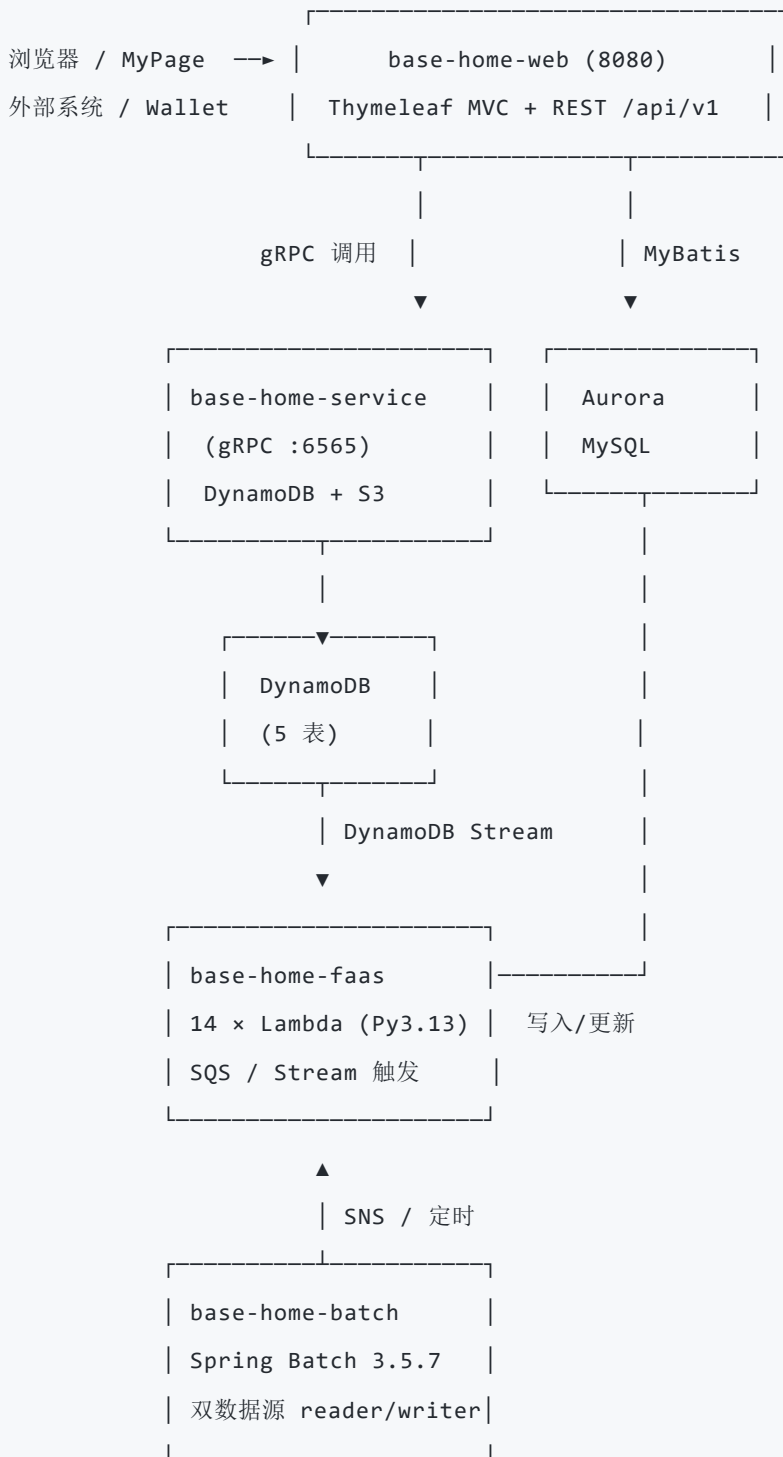
业务域	日文原文	承载模块
预约管理	予約管理	web（预约搜索/管理/全局搜索）+ service（Travel 系列 RPC）
客户消息	顧客メッセージ	web（20+ 消息控制器）+ service（Message RPC 8 个）
员工消息	社員メッセージ	web
通知发送	通知配信	batch（邮件/SNS）+ faas（Wallet 通知、MyPage 通知）
文件管理	ファイル管理	web（S3 下载）+ service（上传/列表 RPC）
亲权同意书	親権同意書	service（Parental 系列）+ faas

业务域	日文原文	承载模块
主数据管理	マスタ管理	batch (MDM 联动) + common (26 张 M 表)
ToDo 管理	ToDo 管理	batch + faas + web
成本文件	コストファイル	batch (ACT/AIR/HTL 三类成本文件生成) + web (下载)

项目规模 (真实)

指标	数量
Gradle 模块数	8 个 (web / service / batch / common / spec / faas / versions / environment)
独立 Git 仓库	各 Java 模块独立仓库, 无顶层聚合
gRPC RPC 方法	17 个 (BaseHomeService), 服务端 18 个实现类
protobuf 文件	约 90 个 (enums 13 + request 17 + response 17 + type 约 42)
MyBatis 实体	79 个 (26 张 M* 主数据表 + 53 张 T* 业务表)
MyBatis Mapper	54 个
AWS Lambda 函数	14 个 (Python 3.13)
DynamoDB 表	5 张 (travel / document / parental / sequence / message_thread)
Spring Batch Job	10 个 (含 1 个测试用 NoopJob)

二、模块清单（架构图的骨架， 真实）



基座: base-home-common (实体/常量/消息/CSV)

base-home-spec (proto → Java/Go/PHP + OpenAPI 文档)

base-home-versions (Gradle 统一版本管理插件)

各模块要点（写作时按主题取用）

模块	技术要点	可支撑的论文主题
base-home-web	Spring Boot MVC + Thymeleaf、Spring Security + OAuth2/OIDC、Redis Cluster Session、MyBatis、SpringDoc /api-docs	安全架构、高可用、系统集成
base-home-service	gRPC 服务端（@GRpcService）、DynamoDB Enhanced Client、S3、STS AssumeRole、重试	微服务、数据架构、集成、高可用
base-home-batch	Spring Batch、双数据源读写分离、SNS、MDM 联动、Prometheus PushGateway、幂等控制	批处理、数据架构、高可用
base-home-common	统一消息体系（EABH00001 类消息 ID）、SuperCSV、79 实体、自动生成的双语 properties	数据架构、复用
base-home-spec	90 个 proto、一份契约生成 Java/Go/PHP 三语言、OpenAPI + MkDocs	系统集成、架构复用、微服务
base-home-faas	14 个 Lambda、Powertools、SQS / DynamoDB Stream 触发、SSM Parameter Store、pytest + moto	云原生、FaaS、数据架构、高可用
base-home-versions	Gradle 插件统一版本 + resolutionStrategy force	架构可维护性、构建治理

三、技术栈版本（ 真实，可直接引用）

组件	版本
Java	21
Spring Boot	3.5.7
Spring Batch	3.5.7
Gradle	8.10 (wrapper) / 8.14.3 (CI)
MyBatis	3.0.5 (强制覆盖 Spring Boot 托管版本)
gRPC (io.grpc)	1.70.0
protobuf	[3,4) 区间，实际 3.25.x
proto-google-common-protos	2.63.2 (强制降级，见难点素材 ②)
Redis 客户端	Jedis (Redis Cluster 6 节点)
AWS SDK	v1 + v2 并存 (见难点素材 ③)
Python	3.13
Go	1.20
PHP	grpc ^1.42.0 / protobuf ^3.19 ^4.26
测试	TestContainers + LocalStack + H2 + embedded-redis + pytest/moto

四、难点素材库（论文「难点与解决」段的核心弹药）

每个难点都已标注**可套用的主题**。考场上优先选与你题目最贴合的 2~3 个。

① 一份 proto 契约驱动三语言客户端

- **问题**：系统需同时为 Java（内部服务）、Go（周边平台）、PHP（既有遗留系统）提供 API。若各自维护接口定义，字段口径必然漂移，联调成本高，回归风险大。
- **方案**：在 `base-home-spec` 中以 protobuf 作为**唯一契约源**，通过 `build.sh` 编排 `build-java → build-go → build-php → generate-all`，分别用 `protoc-gen-grpc-java`、`protoc --go_out/--go-grpc_out/--grpc-gateway_out`、`--php_out + grpc_php_plugin` 生成三语言产物并独立发布（Maven / Go module / Composer）。同时用 `--openapi2_out` 产出 `base_home_api.swagger.json`，经 MkDocs 发布为在线文档。
- **可量化效果**：一份契约、**17 个 RPC** 三端一致；接口变更由构建流水线统一验证，消除了人工同步定义；`base-home-spec` 产出的 90 个 proto 文件全部纳入版本管理。◆ 建议补充：改造前后接口联调耗时对比、字段不一致缺陷数下降比例。
- **可套用主题**：系统集成、微服务、架构复用、架构演化

② protobuf 依赖版本冲突导致服务启动失败

- **问题**：`proto-google-common-protos` 升到 `2.64.0-rc1` 后，gRPC 服务启动报错，且因版本区间声明为 `[3,4)`、`grpcVersion = latest.release`，故障在构建期不显现，只在运行时暴露，属于典型的**依赖漂移**引发的架构风险。
- **方案**：在 `base-home-versions` 的统一版本源中，通过 `resolutionStrategy { force ... }` 将 `proto-google-common-protos` 锁定到 **2.63.2**，并同步锁定 MyBatis 3.0.5 等关键组件，把「隐式版本」变成「显式契约」。
- **可量化效果**：启动失败问题消除；构建可复现（同一 commit 任意机器产物一致）。◆ 建议补充：锁定前每月因依赖漂移导致的构建/启动故障次数。
- **可套用主题**：微服务治理、架构可维护性、架构演化、CI/CD（本难点**非常适合**“架构可维护性/演化”类题目，因为它体现“架构决策的长期成本”）

③ AWS SDK v1 与 v2 长期并存

- **问题**：项目历史遗留使用 AWS SDK **v1**（S3 / STS），而 DynamoDB 增强客户端（`DynamoDbEnhancedClient`）只在 **v2** 提供，两套 SDK 的凭证链、异常模型、配置方式完全不同，直接替换会波及大量既有代码。
- **方案**：**接受双版本并存的过渡态**，按能力边界划分：新增能力（DynamoDB 增强访问、STS AssumeRole）一律走 v2，既有能力暂留 v1；在 `DynamoDbConfig` / `AwsConfig` 中集中封装客户端构建，统一注入 STS AssumeRole 临时凭证 + `DefaultCredentialsProvider` 兜底。
- **可量化效果**：新功能不再受阻于 SDK 版本；凭证获取统一收口到配置类，一账户一集群。◆ 建议补充：统一凭证后的凭证失效类故障下降情况。
- **可套用主题**：架构演化、遗留系统改造、系统集成（这是“**不追求一次性重写、渐进式演进**”的绝佳论据）

④ MySQL 读写分离双数据源

- **问题：**批处理既要从生产库大量读取（成本文件、ToDo、预约导入），又要回写结果，读写互相争抢资源，长事务影响在线业务。
- **方案：**在 `base-home-batch` 中配置**双数据源**：`ReaderDataSourceConfig`（`spring.datasource.reader`）与 `WriterDataSourceConfig`（`spring.datasource.writer`，标 `@Primary`），通过包路径隔离 Mapper（`repository/reader/` 与 `repository/writer/`），事务隔离级别设为 `TRANSACTION_READ_COMMITTED`，`defaultFetchSize=200` 控制批量游标。
- **可量化效果：**读写物理隔离，批量作业不再挤占在线库连接池。◆ 建议补充：作业耗时、在线接口 P99 响应时间的改善数据。
- **可套用主题：**数据架构、批处理、高可用、性能优化

⑤ DynamoDB 单表多实体 + 混合存储的数据架构

- **问题：**旅游业务数据形态差异极大——旅行主数据（Travel）字段频繁演进、文档（Document）为二进制引用、消息线程（MessageThread）读写频繁、序列号（Sequence）要求强一致自增。若全部塞进关系库，扩展性与弹性不足；若全部 NoSQL，又丢失事务与关联查询能力。
- **方案：混合存储（Polyglot Persistence）：**
- 关系型（Aurora MySQL）承载交易强一致与复杂查询（53 张 `T*` 业务表 + 26 张 `M*` 主数据表）
- DynamoDB 承载高弹性、schema-less 的 5 张表，采用 **分区键 + 排序键** 设计（如 `travel` 表以 `travel_no` 为分区键、`data_type` 为排序键、`json_data` 存半结构化 JSON），实现**单表多实体**，避免表数量膨胀
- S3 承载文件实体（`ap-northeast-1` 区域）
- `Sequence` 表用 `seq_key/seq_no` 提供分布式序列号
- **可量化效果：**`scan-limit=10000` 控制扫描边界；表名带环境前缀，实现「1 账户 1 集群」隔离。◆ 建议补充：单表承载的实体种类数、查询延迟数据。
- **可套用主题：**数据架构、大数据、微服务（每个主题都会问“数据怎么存”）

⑥ Serverless 数据同步链路（最终一致性）

- **问题：**DynamoDB（服务端写入）与 MySQL（报表/既有系统查询）之间需要同步，但同步逻辑若放在 Java 服务里，会拉长主链路响应时间，且扩容成本高。
- **方案：事件驱动的 Serverless 同步：**DynamoDB Stream 与 SQS 作为事件源触发 14 个 Lambda 函数，按业务域拆分（`import_travel_information`、`import_document_information`、`import_message_thread`、`import_booking_information`、`import_mail_information`、`import_parental_consent`、`import_notification`、`import_wallet_status` 等），每个函数独立 Dockerfile、独立 GitHub Actions workflow、独立 SAM 部署，配置从 SSM Parameter Store 读取，用 `aws_lambda_powertools` 做日志/链路追踪/批量部分失败处理。主链路只写 DynamoDB 即返回，同步异步化，达成**最终一致性**。
- **可量化效果：**14 个函数按域独立扩缩容；单个函数故障不影响其他业务域。◆ 建议补充：同步延迟（秒级）、峰值消息处理量、按需计费节省的成本比例。
- **可套用主题：**云原生、FaaS、数据架构、高可用（**近两年最热，强烈建议准备**）

⑦ 批量消息推送的幂等与重试

- **问题：**客户消息（顧客メッセージ）常需**批量推送**（`BulkPutCustomerMessage`），邮件发送（`MailSendJob`、`AutomaticEmailCreateJob`）存在网络抖动、第三方邮件网关限流，重试若不加控制会产生**重复发送**，对客户是严重体验事故。
- **方案：**
- 幂等控制：`app.batch.idempotency-duration-minutes=10` 时间窗去重
- 专用重试服务：`MailSendRetryService` 独立承担重试，与业务解耦
- 事务边界：`service/transaction/*TransactionalService` 拆分事务粒度
- 可观测：Prometheus PushGateway（`shutdown-operation: post`）上报作业指标
- **可量化效果：**重复发送被幂等窗口拦截；作业失败可观测、可重跑。◆ 建议补充：重试成功率、重复发送率降到多少。
- **可套用主题：**批处理、高可用、数据一致性

⑧ 跨国多时区时间一致性

- **问题：**系统服务日本本部（源库为 **JST**）、部署环境与开发终端时区各异（容器为 **UTC**、开发机为 **UTC+8**）。预约时间、截止时间、邮件发送时刻等一旦跨时区比较出错，会出现「提前发送」或「漏发」——对旅游业务是不可接受的。
- **方案：**确立**时间口径单一化原则**：存储统一 **UTC**，展示按业务方时区（**JST**）转换，转换动作集中到 `FormatUtils` 等工具类，禁止在业务代码中散落 `LocalDateTime.now()`；批处理的 `executionBaseDate`、`acquisitionRange` 作为 `JobParameter` 显式传入，不依赖系统时钟。
- **可量化效果：**时间比较错误集中收口；批处理基准日期可重放（同一参数结果可复现）。◆ 建议补充：改造前时区相关缺陷单数量。
- **可套用主题：**数据架构、批处理、国际化系统架构（**特色素材，别人写不出来，容易出彩**）

⑨ 分布式会话与在线扩容

- **问题：**Web 层多实例部署后，Session 若存本地内存，用户会被反复踢下线。
- **方案：**引入 **Spring Session Data Redis**，接 **Redis Cluster（6 节点）**，客户端 Jedis；`spring.session.timeout=3600`。安全侧配合 `spring-session` 与 Spring Security 的会话并发控制。
- **可量化效果：**Web 实例无状态化，可水平扩容；实例重启不掉线。◆ 建议补充：扩容后支撑的并发用户数、会话序列化后的请求延迟。
- **可套用主题：**高可用、高性能、微服务

⑩ 主数据统一（MDM）与外部系统联动

- **问题：**H.I.S. 集团存在 MDM（Master Data Management）主数据平台，本国、城市、航空公司、货币等主数据需与 BASE HOME 保持一致，人工维护必然不一致。
- **方案：**`ImportMdmJob` 定时从 MDM API 拉取主数据落库，经 `api/mdm/` 封装认证与查询参数；本地保留 26 张 `M*` 主数据表作为只读副本，业务查询不直连外部。
- **可量化效果：**主数据一致性由作业保障，业务侧零改造。◆ 建议补充：主数据同步频率、覆盖的主数据表数量。

- **可套用主题：**系统集成、数据架构、企业应用集成（EAI）

五、论文通用段落（背下来，考场上省 20 分钟）

5.1 摘要万能结构（约 300 字）

本文论述了本人参与的【项目名称】的【主题】设计与实现。
该系统是【一句话业务定位】，服务于【用户群体】，
采用【核心技术栈】构建，本人担任【角色，务必写“架构师/技术负责人”】，
负责【3 项具体职责】。
针对【核心难题 1】、【核心难题 2】两大挑战，
本文从【架构视角 1】、【架构视角 2】、【架构视角 3】三个维度进行设计，
通过【关键手段】实现了【量化成果 1】和【量化成果 2】。
系统上线后【运行效果】，验证了方案的有效性。

摘要三要素：项目背景 + 本人角色 + 主要工作与成效。结尾必须落到「成效」，阅卷老师主要看这一段判断你有没有真做过。

5.2 项目背景段模板（约 400 字）

【项目名称】是【公司名】面向【用户群体】建设的【系统定位】，
覆盖【业务域 1】、【业务域 2】、【业务域 3】等核心业务，
日均处理【◆ 填真实量级】。
系统自【◆ 时间】启动建设，本人作为架构师参与了整体架构设计，
主要负责【你的真实职责】。
项目采用【Java 21 + Spring Boot 3.5.7】技术栈，
按【多模块 Gradle 结构】划分为 Web、Service、Batch、Common、Spec、FaaS 等模块，
部署于 AWS【区域】。

5.3 结尾段模板（约 200 字）

目前该系统已稳定运行【◆ 时长】，支撑【◆ 业务量】，
【核心指标】达到【◆ 数值】。
回顾整个建设过程，仍有可改进之处：
一是【不足 1】，二是【不足 2】。
后续计划通过【改进方向】进一步优化。
通过本项目，本人深刻体会到
【一句话架构感悟，如：架构设计的本质是在质量属性之间做权衡】。

结尾必须写「不足」。不写不足的论文常被认为缺乏架构思维。不足要写得「真诚但无害」，例如「跨语言契约的破坏性变更尚未实现自动化兼容性检测」，而不是「系统经常宕机」。

六、论文写作红线（阅卷扣分点）

红线	说明
✗ 字数不足	少于 2000 字基本判不及格，必须练到稳定 2500 ~ 3000 字
✗ 通篇讲原理	架构师论文考「你的决策」，不是考「什么是微服务」
✗ 没有量化	难点必须有数字，没有数字的"难点"是伪难点
✗ 不写本人角色	「我们团队」是减分项，要写「我主导了...我选择了...」
✗ 不写权衡	只写「我选了 A」不写「为什么不是 B」，是初级工程师视角
✗ 摘要与正文脱节	摘要里提到的三个维度，正文必须有对应小标题
✗ 技术堆砌无逻辑	列出 10 个技术名词却不解释它们如何协同，是典型外行写法
✓ 加分项	写架构演化过程（从 A 演进到 B，为什么）、写失败与修正

七、◆ 必须由你补充的「数字清单」

论文想拿高分，下面这些数据必须补全。请对照真实情况填写，**不要编造**：

类别	需要的数据	建议来源
业务规模	日均预约量、注册旅行者数、消息日推量	运营报表
用户规模	营业本部使用人数、门店数、并发用户数	业务方
性能	接口 P99 响应时间、批处理作业耗时、同步延迟	APM / Prometheus
可用性	SLA 目标、年度可用率、故障恢复时间（MTTR）	运维记录
容量	数据库数据量、DynamoDB 表条目数、S3 存储量	云监控
成本	上云前后成本对比、Serverless 节省比例	财务 / Cost Explorer
质量	缺陷率下降、重复发送率、数据不一致单数	质量报告
时间	项目启动时间、上线时间、稳定运行时长	项目计划

写作技巧：数据可以不精确，但必须**自洽**。例如说「日均 3 万笔预约」，后文就不能出现「日处理 300 笔」。建议先在纸上把口径统一，再动笔写正文。